

Università degli Studi di Torino
Corso di Laurea Magistrale in Informatica

Tecnologie del Linguaggio Naturale

Parte 3 – Prof. Luigi Di Caro

Lab 5

Hybrid Search RAG

Retrieval-Augmented Generation con ricerca ibrida semantica e lessicale

Gruppo di lavoro:

Studente	Matricola
Alessandro Olivero	915069
Samuele Iudice	1235245
Andrea Franco	1226739

Docente: Prof. Luigi Di Caro

Anno Accademico: 2025/2026

Indice

1	Introduzione	2
2	Background Teorico	2
2.1	Retrieval-Augmented Generation (RAG)	2
2.2	Retrieval semantico con FAISS e SciBERT	2
2.3	Retrieval lessicale con BM25	3
2.4	Reciprocal Rank Fusion (RRF)	3
2.5	Phi-3.5-mini-instruct	3
2.6	Metriche di valutazione ispirate a RAGAS	3
3	Architettura del Sistema	4
4	Dataset e Configurazione	5
4.1	Dataset	5
4.2	Configurazione dell'esperimento	6
4.3	Modelli utilizzati	6
5	Implementazione del Retrieval	7
5.1	Retrieval semantico	7
5.2	Retrieval BM25	7
6	Generazione della Risposta	8
7	Metriche di Valutazione: Implementazione	8
8	Esperimento: Variazione di N_DOCS	9
8.1	Risultati per singolo N_DOCS	9
8.2	Riepilogo comparativo: Precision@5	11
8.3	Dettaglio per query: Precision@5, base vs. ibrido	11
8.4	Robustezza al variare di N_DOCS	11
9	Risultati e Discussione	11
9.1	Il retrieval ibrido migliora sistematicamente la Precision@5	11
9.2	Il crollo del sistema base sul corpus completo	12
9.3	Andamento non monotono della Precision@5 con N_DOCS	12
9.4	Context Relevance e Answer Relevance: differenze minime	12
9.5	Faithfulness: il sistema ibrido produce risposte più fedeli	12
9.6	Robustezza: non è più una vittoria automatica dell'ibrido	13
9.7	Query con performance diverse	13
9.8	Nota metodologica: correzione applicata	14
10	Conclusioni	14

1 Introduzione

Questo laboratorio affronta il problema della **Retrieval-Augmented Generation (RAG)**, un paradigma che potenzia i modelli di linguaggio combinando un meccanismo di *retrieval* su una knowledge base esterna con la capacità generativa di un Large Language Model (LLM). L'obiettivo è confrontare due strategie di retrieval su un corpus scientifico NLP:

- **RAG base**: recupero puramente semantico tramite embeddings SciBERT e indice FAISS.
- **RAG ibrido**: combinazione del retrieval semantico con BM25 lessicale, fusi tramite Reciprocal Rank Fusion (RRF).

L'esperimento principale varia la dimensione del corpus da 1 000 a 44 949 documenti, misurando l'impatto sulle prestazioni con quattro metriche ispirate al framework RAGAS: Precision@5, Context Relevance, Answer Relevance e Faithfulness. Si analizza inoltre la **robustezza** del sistema a parafrasature delle query.

Il dataset usato è **MaartenGr/arxiv_nlp**, una collezione di paper scientifici NLP da arXiv, disponibile su Hugging Face. Il modello di linguaggio è **Phi-3.5-mini-instruct** di Microsoft, eseguito interamente in locale su CPU.

2 Background Teorico

2.1 Retrieval-Augmented Generation (RAG)

Un sistema RAG si articola in tre componenti:

1. **Retriever**: dato una query, recupera i documenti più rilevanti da una knowledge base.
2. **Context assembly**: i documenti recuperati vengono concatenati come contesto.
3. **Generator**: un LLM produce una risposta condizionata dal contesto e dalla query.

Il vantaggio principale rispetto a un LLM stand-alone è la capacità di accedere a informazioni aggiornate e specifiche di dominio senza ri-addestrare il modello, riducendo al contempo le allucinazioni.

2.2 Retrieval semantico con FAISS e SciBERT

Il retrieval semantico si basa su **dense embeddings**: ogni documento e ogni query vengono proiettati in uno spazio vettoriale denso, e il retrieval è una ricerca dei vicini più prossimi (Nearest Neighbor Search).

Si usa **SciBERT** (`allenai/scibert_scivocab_uncased`) come modello di embedding: un BERT pre-addestrato su 1,14 milioni di paper scientifici da Semantic Scholar. Produce vettori di dimensione 768, normalizzati a norma unitaria per l'uso con similarità coseno.

Nota sulla scelta del modello: SciBERT non è un modello **sentence-transformers** nativo, cioè non è stato fine-tuned con un obiettivo di similarità tra frasi/documenti; caricandolo con `SentenceTransformer(...)` si ottiene un mean pooling di default (con relativo

warning a runtime), che funziona ma può soffrire del problema noto di *anisotropia* degli embedding BERT non specializzati per il retrieval. La scelta resta difendibile per il vocabolario specializzato nel dominio scientifico, ma un'alternativa più indicata sarebbe `allenai/specter` (o `specter2`), una variante di SciBERT già addestrata esplicitamente per la similarità tra paper scientifici (tramite citazioni come segnale di supervisione).

L'indice di ricerca è costruito con **FAISS** (Facebook AI Similarity Search), che implementa **IndexFlatIP** (inner product su vettori normalizzati, equivalente alla similarità coseno) e consente ricerca efficiente su decine di migliaia di vettori.

2.3 Retrieval lessicale con BM25

BM25 (Best Match 25) è un modello di recupero probabilistico basato sulla frequenza dei termini. Per ogni query q e documento d , il punteggio è:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)}$$

dove $f(t, d)$ è la frequenza del termine t nel documento d , $|d|$ è la lunghezza del documento, avgdl è la lunghezza media del corpus, e $k_1 = 1.5$, $b = 0.75$ sono iperparametri standard.

A differenza del retrieval semantico, BM25 è efficace per query con termini tecnici specifici (es. acronimi, nomi propri) che gli embeddings tendono a generalizzare.

2.4 Reciprocal Rank Fusion (RRF)

La **Reciprocal Rank Fusion** è una tecnica di fusione dei risultati di più retriever. Dati i ranking prodotti da n retriever, il punteggio RRF di un documento d è:

$$\text{RRF}(d) = \sum_{r=1}^n \frac{1}{k + \text{rank}_r(d)}$$

con $k = 60$ (costante di smorzamento). I documenti vengono ri-ordinati per punteggio RRF decrescente. Questo approccio è robusto perché non richiede normalizzazione degli score eterogenei e valorizza i documenti consistentemente rilevanti per più retriever.

2.5 Phi-3.5-mini-instruct

Il modello generativo è **Phi-3.5-mini-instruct** di Microsoft: un LLM da 3.8B parametri ottimizzato per il ragionamento e l'instruction-following. Viene eseguito in modalità *float32* su CPU, con generazione limitata a 300 token. Il template di chat segue il formato Phi-3 con ruoli `system` e `user`.

2.6 Metriche di valutazione ispirate a RAGAS

Si implementano quattro metriche per valutare il sistema RAG, ispirate al framework RAGAS (Retrieval-Augmented Generation Assessment System):

- **Precision@5**: percentuale di documenti rilevanti (per gold standard) tra i top-5 recuperati.

$$P@5 = \frac{|\{d \in R_5 : d \text{ rilevante}\}|}{5}$$

- **Context Relevance (CR)**: similarità coseno media tra l'embedding SciBERT della query e gli snippet dei documenti recuperati. Misura quanto il contesto è semanticamente pertinente alla domanda.

$$CR = \frac{1}{|R|} \sum_{d \in R} \cos(\text{enc}(q), \text{enc}(d[:200]))$$

- **Answer Relevance (AR)**: similarità coseno tra l'embedding della query e l'embedding della risposta generata dall'LLM. Misura se la risposta è centrata sulla domanda.

$$AR = \cos(\text{enc}(q), \text{enc}(a))$$

- **Faithfulness proxy (FF)**: percentuale di parole contenutistiche della risposta (escluse le stopwords) che compaiono nel testo dei documenti recuperati. Misura in modo approssimato quanto la risposta è fondata sul contesto.

$$FF = \frac{|\{w \in a_{\text{content}} : w \in C\}|}{|a_{\text{content}}|}$$

dove C è il testo concatenato dei documenti di contesto.

- **Robustezza**: per un sottoinsieme di query, si calcolano le parafrasature e si misura l'overlap@5 tra i risultati della query originale e quelli delle parafrasature:

$$\text{Rob} = \frac{1}{|P|} \sum_{p \in P} \frac{|R_5(q) \cap R_5(p)|}{5}$$

Un valore alto indica che il retrieval è stabile a variazioni di formulazione della query.

3 Architettura del Sistema

Il sistema si struttura in tre livelli:

1. **Livello dati**: caricamento del dataset arXiv-NLP, campionamento di N documenti, costruzione del testo come concatenazione di titolo e abstract.
2. **Livello retrieval**: indice FAISS (semantico) + indice BM25 (lessicale), con fusione RRF per il retrieval ibrido.
3. **Livello generazione**: Phi-3.5-mini-instruct riceve la query e i top-5 documenti come contesto e genera una risposta in linguaggio naturale con citazioni.

Funzione di retrieval ibrido con RRF

```
1 def retrieve_hybrid(query, faiss_index, bm25, documents, metadata, k=5):
2     # retrieval semantico (top-20) e lessicale (top-20)
3     sem = retrieve_semantic(query, faiss_index, documents, metadata, k=20)
4     bm = retrieve_bm25(query, bm25, documents, metadata, k=20)
5     # fusione RRF e restituzione dei top-k
6     return reciprocal_rank_fusion([sem, bm][:k])
7
8 def reciprocal_rank_fusion(results_list, k=60):
9     rrf_scores, doc_map = {}, {}
10    for results in results_list:
11        for item in results:
12            ix = item['idx']
13            rrf_scores[ix] = rrf_scores.get(ix, 0) + 1 / (k + item['rank'])
14            doc_map[ix] = item
15    sorted_docs = sorted(rrf_scores.items(),
16                        key=lambda x: x[1], reverse=True)
17    return [
18        {'rank': i+1, 'rrf_score': score, 'idx': ix,
19         'document': doc_map[ix]['document'],
20         'metadata': doc_map[ix]['metadata']}
21        for i, (ix, score) in enumerate(sorted_docs)
22    ]
```

Il retrieval ibrido parte da $k' = 20$ risultati per ciascun retriever prima della fusione, così da avere un pool di candidati sufficientemente ampio per il re-ranking.

4 Dataset e Configurazione

4.1 Dataset

Il dataset MaartenGr/arxiv_nlp contiene **44 949 paper scientifici NLP** da arXiv, con i campi title, abstract e year. Ogni documento è rappresentato come la concatenazione di titolo e abstract.

Caricamento e costruzione del corpus

```
1 dataset = load_dataset("MaartenGr/arxiv_nlp")
2 df_full = pd.DataFrame(dataset['train'])
3 # Documenti totali disponibili: 44,949
4
5 def build_corpus(df, n):
6     df_s = df.sample(n=n, random_state=42).reset_index(drop=True)
7     docs, meta = [], []
8     for _, row in df_s.iterrows():
9         title = str(row['title'])
10        abstract = str(row['abstract'])
11        docs.append(f>Title: {title}\n\nAbstract: {abstract}")
12        meta.append({'title': title, 'year': str(row['year'])})
13    return docs, meta
```

4.2 Configurazione dell’esperimento

L’esperimento varia la dimensione del corpus su cinque livelli:

$$N_{\text{DOCS}} \in \{1\,000, 5\,000, 15\,000, 25\,000, 44\,949\}$$

Per ogni configurazione si ricostruisce il corpus (campionamento casuale con seed fisso), si riaddestrano gli indici FAISS e BM25, e si rieseguo le valutazioni. Il gold standard è definito da cinque query NLP con parole chiave rilevanti:

Tabella 1: Query di valutazione e parole chiave del gold standard

Query	Parole chiave rilevanti
What is BERT and how does self-attention work?	bert, attention, transformer, language model, pre-training
How does word2vec represent word meaning?	word2vec, skip-gram, word embedding, word representation
What are the main challenges in machine translation?	machine translation, neural machine translation, nmt
How does topic modeling work?	topic model, lda, bertopic, topic, clustering
What is named entity recognition?	named entity, ner, entity recognition, sequence labeling

4.3 Modelli utilizzati

Tabella 2: Modelli e librerie del sistema

Componente	Modello/Libreria	Ruolo
Embedding	SciBERT (<code>allenai/scibert_scivocab_uncased</code>)	Retrieval semantico
Indice vettoriale	FAISS <code>IndexFlatIP</code>	ANN search
Retrieval lessicale	BM25Okapi (<code>rank_bm25</code>)	Keyword search
Fusione	Reciprocal Rank Fusion ($k = 60$)	Re-ranking ibrido
Generazione	Phi-3.5-mini-instruct (3.8B)	Produzione risposta

5 Implementazione del Retrieval

5.1 Retrieval semantico

Costruzione indice FAISS e retrieval semantico

```
1 def build_faiss_index(documents):
2     embeddings = embedding_model.encode(
3         documents, show_progress_bar=True,
4         batch_size=64, convert_to_numpy=True,
5         normalize_embeddings=True           # norma L2 = 1 (inner product =
6         coseno)
7     )
8     idx = faiss.IndexFlatIP(embeddings.shape[1]) # dimensione 768
9     idx.add(embeddings.astype(np.float32))
10    return idx
11
12 def retrieve_semantic(query, faiss_index, documents, metadata, k=10):
13     qe = embedding_model.encode(
14         [query], convert_to_numpy=True, normalize_embeddings=True
15     ).astype(np.float32)
16     sims, idxs = faiss_index.search(qe, k)
17     return [
18         {'rank': i+1, 'score': float(s), 'idx': int(ix),
19          'document': documents[ix], 'metadata': metadata[ix]}
20         for i, (s, ix) in enumerate(zip(sims[0], idxs[0]))
21     ]
```

Il retrieval semantico codifica la query con SciBERT e recupera i k documenti con massima similarità coseno. FAISS `IndexFlatIP` calcola l'inner product su tutti i vettori: essendo normalizzati, coincide esattamente con la similarità coseno.

5.2 Retrieval BM25

Costruzione indice BM25 e retrieval lessicale

```
1 def build_bm25_index(documents):
2     tokenized = [doc.lower().split() for doc in documents]
3     return BM25Okapi(tokenized), tokenized
4
5 def retrieve_bm25(query, bm25, documents, metadata, k=10):
6     scores = bm25.get_scores(query.lower().split())
7     top    = np.argsort(scores)[::-1][:k]
8     return [
9         {'rank': i+1, 'score': float(scores[ix]), 'idx': int(ix),
10          'document': documents[ix], 'metadata': metadata[ix]}
11         for i, ix in enumerate(top)
12     ]
```

BM25 tokenizza i documenti con semplice split su spazio e punteggia ogni documento in base alla frequenza normalizzata dei termini della query. Non richiede inferenza neurale, quindi è ordini di grandezza più veloce del retrieval semantico a runtime.

6 Generazione della Risposta

Prompt e generazione con Phi-3.5-mini-instruct

```
1 def rag_answer(query, results):
2     context_parts = [
3         f"[Paper {r['rank']}] {r['metadata']['title']}\n{r['document']}"
4         for r in results
5     ]
6     context = "\n\n".join(context_parts)
7     if len(context) > 3000:
8         context = context[:3000] + "\n[...truncated...]"
9
10    messages = [
11        {"role": "system", "content": (
12            "You are a helpful assistant specialized in NLP research. "
13            "Answer questions based only on the provided research papers. "
14            "Always cite the paper numbers [Paper X] you used. "
15            "If the context is insufficient, say so clearly."
16        )},
17        {"role": "user", "content": (
18            f"Context from research papers:\n\n{context}\n\n"
19            f"Question: {query}\n\n"
20            f"Answer based on the context above, citing paper numbers:"
21        )}
22    ]
23    prompt = tokenizer.apply_chat_template(
24        messages, tokenize=False, add_generation_prompt=True
25    )
26    output = llm(prompt)
27    full_text = output[0]['generated_text']
28    return full_text[len(prompt):].strip()
```

Il prompt è strutturato con ruolo `system` (vincola l'LLM all'uso del solo contesto fornito e richiede citazioni esplicite) e ruolo `user` (context + query). Il contesto è troncato a 3 000 caratteri per gestire la finestra di contesto del modello su CPU. La temperatura è $T = 0.3$ per favorire risposte deterministiche.

7 Metriche di Valutazione: Implementazione

Context Relevance e Answer Relevance

```
1 def context_relevance(query, results):
2     qe = embedding_model.encode(
3         [query], convert_to_numpy=True, normalize_embeddings=True)
4     snip = [r['document'][:200] for r in results]
5     de = embedding_model.encode(
6         snip, convert_to_numpy=True, normalize_embeddings=True)
7     return float(np.mean(de @ qe.T)) # media dei coseni query-snippet
8
9 def answer_relevance(query, answer):
10    embs = embedding_model.encode(
11        [query, answer], convert_to_numpy=True, normalize_embeddings=True)
12    return float(embs[0] @ embs[1]) # coseno query-risposta
```

```

1 def faithfulness_proxy(answer, results):
2     context_text = ' '.join(r['document'][:500] for r in results).lower()
3     words = re.findall(r'\b[a-z]{4,}\b', answer.lower())
4     content_words = [w for w in words if w not in sw] # sw = stopwords NLTK
5     if not content_words:
6         return 0.0
7     return sum(1 for w in content_words if w in context_text) / len(
        content_words)
8
9 def robustness_test(query, paraphrases, faiss_index, bm25,
10                     documents, metadata, k=5):
11     base_titles = {r['metadata']['title']}
12     for r in retrieve_hybrid(
13         query, faiss_index, bm25, documents, metadata, k):
14     overlaps = []
15     for para in paraphrases:
16         para_titles = {r['metadata']['title']}
17         for r in retrieve_hybrid(
18             para, faiss_index, bm25, documents, metadata, k):
19             overlaps.append(len(base_titles & para_titles) / k)
20     return float(np.mean(overlaps))

```

La **faithfulness proxy** è una stima lessicale della fedeltà: controlla che le parole contenutistiche della risposta siano ancorate al testo dei documenti recuperati. Non misura la correttezza logica (richiederebbe un NLI model), ma individua risposte “fuori contesto”.

La **robustezza** testa tre query con due parafrasature ciascuna, misurando l’overlap@5 tra i risultati della query originale e quelli delle varianti. Un overlap basso segnala instabilità del retriever a piccole variazioni lessicali.

8 Esperimento: Variazione di N_DOCS

8.1 Risultati per singolo N_DOCS

Le tabelle seguenti riportano i risultati completi per ciascuna configurazione (dati reali, esecuzione corrente con subset annidati e metodologia corretta, si veda la nota in Sezione 9.8).

Tabella 3: Risultati N_DOCS = 1000

Metrica	Base	Ibrido	Delta
Precision@5	0.160	0.480	+0.320
Context Relevance	0.716	0.714	−0.002
Answer Relevance	0.676	0.657	−0.019
Faithfulness	0.568	0.765	+0.197
Robustezza	0.233	0.267	+0.033

Tabella 4: Risultati N_DOCS = 5000

Metrica	Base	Ibrido	Delta
Precision@5	0.280	0.680	+0.400
Context Relevance	0.724	0.722	−0.002
Answer Relevance	0.676	0.659	−0.017
Faithfulness	0.613	0.719	+0.106
Robustezza	0.167	0.100	−0.067

Tabella 5: Risultati N_DOCS = 15000

Metrica	Base	Ibrido	Delta
Precision@5	0.200	0.560	+0.360
Context Relevance	0.713	0.731	+0.018
Answer Relevance	0.683	0.675	−0.008
Faithfulness	0.655	0.755	+0.100
Robustezza	0.100	0.067	−0.033

Tabella 6: Risultati N_DOCS = 25000

Metrica	Base	Ibrido	Delta
Precision@5	0.200	0.560	+0.360
Context Relevance	0.720	0.728	+0.008
Answer Relevance	0.692	0.667	−0.025
Faithfulness	0.628	0.722	+0.094
Robustezza	0.067	0.000	−0.067

Tabella 7: Risultati N_DOCS = 44949 (corpus completo)

Metrica	Base	Ibrido	Delta
Precision@5	0.040	0.480	+0.440
Context Relevance	0.719	0.724	+0.005
Answer Relevance	0.687	0.680	−0.007
Faithfulness	0.496	0.669	+0.173
Robustezza	0.033	0.033	0.000

8.2 Riepilogo comparativo: Precision@5

Tabella 8: Precision@5 al variare di N_DOCS

N_DOCS	Base	Ibrido	Delta	Miglioramento
1 000	0.160	0.480	+0.320	+200.0%
5 000	0.280	0.680	+0.400	+142.9%
15 000	0.200	0.560	+0.360	+180.0%
25 000	0.200	0.560	+0.360	+180.0%
44 949	0.040	0.480	+0.440	+1100.0%

8.3 Dettaglio per query: Precision@5, base vs. ibrido

Tabella 9: P@5 per singola query al variare di N_DOCS – Base / Ibrido

Query	1K	5K	15K	25K	44.9K
BERT / self-attention	0.0/0.6	0.2/0.4	0.0/0.4	0.2/0.4	0.0/0.4
word2vec	0.2/0.4	0.2/0.8	0.4/0.6	0.4/0.4	0.0/0.4
machine translation	0.2/0.4	0.4/0.8	0.4/0.6	0.2/0.8	0.0/0.4
topic modeling	0.0/0.4	0.0/0.4	0.0/0.4	0.0/0.4	0.0/0.4
named entity recog.	0.4/0.6	0.6/1.0	0.2/0.8	0.2/0.8	0.2/0.8
Media	0.16/0.48	0.28/0.68	0.20/0.56	0.20/0.56	0.04/0.48

8.4 Robustezza al variare di N_DOCS

Tabella 10: Robustezza (overlap@5 su query parafrasate): base vs. ibrido

N_DOCS	Base	Ibrido
1 000	0.233	0.267
5 000	0.167	0.100
15 000	0.100	0.067
25 000	0.067	0.000
44 949	0.033	0.033

9 Risultati e Discussione

9.1 Il retrieval ibrido migliora sistematicamente la Precision@5

Il dato più netto è che il sistema ibrido supera il sistema base in *tutte e cinque* le configurazioni di N_DOCS, con miglioramenti che vanno da +142.9% a +1100%. L'ibrido porta la Precision@5 media da valori tra 0.040 e 0.280 a valori tra 0.480 e 0.680: il salto è ancora più netto rispetto a quanto misurato nella prima esecuzione (Sezione 9.8), perché ora il confronto è fatto a parità di corpus fra le due configurazioni.

Questo conferma l'ipotesi di partenza: BM25, catturando i termini tecnici esatti (es. “NER”, “LDA”, “skip-gram”), recupera documenti che il solo retrieval semantico manca, perché SciBERT tende a generalizzare i termini tecnici verso concetti più astratti. La fusione RRF bilancia i punti di forza dei due retriever.

9.2 Il crollo del sistema base sul corpus completo

Il risultato più netto di questa esecuzione è il crollo della Precision@5 base a **0.040** sull'intero corpus (44 949 documenti): guardando il dettaglio per query (Tabella 7), a quella dimensione il retrieval puramente semantico non trova *nessun* documento rilevante nei top-5 per 4 query su 5 (BERT/self-attention, word2vec, machine translation, topic modeling azzerate), e recupera qualcosa solo per “named entity recognition” (0.2). Il sistema ibrido, sullo stesso corpus, resta invece a **0.480** – pressoché invariato rispetto alle configurazioni più piccole. Questo è ora un confronto pulito: gli stessi 44 949 documenti, la stessa query, lo stesso gold standard, l'unica differenza è la presenza di BM25 nella fusione. Il retrieval solo-embedding, su un corpus molto ampio e con query brevi e generiche, perde quasi completamente la capacità di isolare i pochi documenti realmente pertinenti tra le decine di migliaia disponibili; BM25 mantiene invece il segnale sulle parole chiave indipendentemente da quanti documenti competono nell'indice.

9.3 Andamento non monotono della Precision@5 con N_DOCS

Sia la curva base sia quella ibrida hanno un picco a N=5 000 (0.280 e 0.680) e calano dopo: il corpus a 5 000 documenti è abbastanza grande da contenere paper rilevanti, ma abbastanza piccolo da non essere “diluito” da troppi documenti distanti dal tema della query. Il calo successivo è però molto più marcato per il sistema base (fino a 0.040) che per l'ibrido (che si stabilizza a 0.480–0.560): la fusione con BM25 non elimina l'effetto del corpus grande, ma ne assorbe gran parte.

9.4 Context Relevance e Answer Relevance: differenze minime

La Context Relevance (CR) e l'Answer Relevance (AR) mostrano differenze piccole tra base e ibrido (tipicamente $|\Delta| < 0.02$, tranne un paio di casi fino a 0.025). Questo è atteso: entrambe le metriche sono calcolate con gli stessi embeddings SciBERT usati anche per il retrieval semantico, quindi condividono in parte lo stesso “punto di vista” – non sono una misura indipendente della qualità del sistema, un limite già dichiarato in partenza e confermato dai dati.

L'AR ibrida è quasi sempre leggermente inferiore a quella base: i documenti trovati da BM25 sono meno “vicini” alla query nello spazio degli embedding (li trova per parole chiave, non per significato), quindi il contesto che l'LLM riceve, pur più corretto nel contenuto, può produrre risposte con embedding leggermente più lontani dalla query.

9.5 Faithfulness: il sistema ibrido produce risposte più fedeli

La Faithfulness è sistematicamente più alta per il sistema ibrido in tutte le configurazioni, con un margine molto più ampio di quanto misurato in precedenza (+0.094 a +0.197, contro +0.027/+0.095 della prima esecuzione). Il salto più grande è proprio a N=1 000 (+0.197) e sul corpus completo (+0.173): i documenti trovati dall'ibrido restano più

legati al vocabolario tecnico della domanda, quindi l’LLM usa più parole che compaiono davvero nei testi forniti, invece di “riempire i vuoti” con parole plausibili ma non presenti nel contesto.

9.6 Robustezza: non è più una vittoria automatica dell’ibrido

Con il dato del sistema base ora disponibile per il confronto (Tabella 8), la robustezza racconta una storia più sfumata di quanto sembrasse: l’ibrido è più robusto del base solo a $N=1000$ (0.267 contro 0.233); a $N=5000$, 15000 e 25000 è il sistema **base** a risultare più stabile alle parafrasi (es. 0.167 contro 0.100 a $N=5000$), e a $N=44949$ i due sistemi sono pari (0.033). In altre parole, aggiungere BM25 non stabilizza automaticamente il retrieval rispetto a piccole variazioni della formulazione della query – anzi, introducendo un secondo canale di recupero basato su corrispondenza esatta delle parole, l’ibrido può diventare più sensibile a un sinonimo diverso nella parafrasi, che BM25 semplicemente non riconosce come la stessa parola chiave. In entrambi i sistemi, comunque, la robustezza cala con corpus più grandi: con più documenti candidati, piccole variazioni lessicali della query spostano più facilmente quali documenti finiscono nei top-5.

9.7 Query con performance diverse

La query “named entity recognition” resta quella con $P@5$ più alta e stabile in versione ibrida (0.6–1.0): il termine “NER” è molto specifico e BM25 lo gestisce perfettamente. La query “topic modeling” è invece la più debole in assoluto: $P@5$ base sempre a 0.0 e ibrida ferma a 0.4 in ogni configurazione, segno che né il retrieval semantico né BM25 trovano bene i pochi paper specificamente su LDA/BERTopic in questo corpus. La query su BERT resta un caso instabile anche nella versione ibrida (0.4 quasi ovunque, mai sopra 0.6): un limite osservato concretamente anche nelle risposte generate (si veda l’esempio sotto).

Esempio – risposta generata sulla query BERT, corpus completo (44 949 doc.)

Base: “Based on the provided research papers, there is no direct information or discussion about BERT (Bidirectional Encoder Representations from Transformers) or self-attention mechanisms. The papers focus on various aspects of Natural Language Processing (NLP)....”

Ibrido: “Based on the provided research papers, there is no direct explanation of BERT or self-attention mechanisms. The papers discuss various topics related to natural language processing (NLP), such as attention in neural machine translation (Paper 2), cross-lingual syntactic ability of multilingual BERT (Paper 4)....”

In entrambi i casi l’LLM dichiara correttamente l’assenza di un paper specificamente dedicato a BERT invece di inventare una risposta – un comportamento di fedeltà al contesto corretto, ma che conferma il limite: nemmeno l’ibrido, su questo corpus e con questo gold standard, recupera un paper che spieghi BERT direttamente; l’ibrido arriva però più vicino al tema (cita esplicitamente un paper su BERT multilingue).

9.8 Nota metodologica: correzione applicata

Una prima esecuzione di questo esperimento presentava alcuni limiti metodologici, poi corretti nel codice (`lab5rag.py` e `lab5_hybrid.py`) prima di generare i numeri riportati sopra:

- **Campionamento non annidato:** ogni corpus a N_DOCS diverso era ottenuto con una chiamata indipendente a `df.sample(...)`, quindi un paper poteva comparire nel corpus da 5 000 documenti e non in quello da 44 949 (o viceversa) per puro caso, non per un vero effetto della dimensione del corpus. Corretto mischiando il dataset una sola volta e facendo prendere a ogni corpus i primi N documenti di quell'ordine fisso, così che i corpus più piccoli siano sempre sottoinsiemi di quelli più grandi.
- **Precision@k non uniforme fra i due script:** uno cercava le parole chiave in titolo+abstract, l'altro solo nel titolo. Corretto usando lo stesso criterio in entrambi.
- **Corrispondenza troppo permissiva:** parole chiave come “ner” o “bert” potevano matchare per errore parole diverse (“corner”, “RoBERTa”). Corretto cercando la parola (o frase) intera.
- **Generazione non deterministica:** veniva usato il campionamento casuale nella generazione, quindi risposte e metriche potevano cambiare a ogni esecuzione. Corretto con generazione greedy (sempre la parola più probabile), riproducibile.

Il fix più importante è il primo: senza subset annidati, la curva “Precision@5 al variare di N_DOCS ” non era interpretabile, perché un calo poteva dipendere solo da quali paper erano finiti per caso in un campione. Con i subset annidati, il crollo del sistema base a $N=44\,949$ (Sezione 11.2) è ora un risultato solido: lo stesso identico corpus, con e senza BM25, produce una differenza di quasi 12 volte in Precision@5.

10 Conclusioni

Il laboratorio ha implementato un sistema RAG ibrido su un corpus di paper scientifici NLP, dimostrando che la combinazione di retrieval semantico (SciBERT + FAISS) e lessicale (BM25) tramite Reciprocal Rank Fusion porta a miglioramenti consistenti e ora misurati in modo pulito (subset annidati, criteri di valutazione uniformi, generazione deterministica).

I risultati principali sono:

1. **Il retrieval ibrido migliora la Precision@5 in modo sistematico:** il delta medio sulle cinque configurazioni è +0.376, con un miglioramento percentuale che va da +142.9% a +1100%.
2. **Il sistema base crolla su corpus grandi, l'ibrido no:** a 44 949 documenti la Precision@5 base scende a 0.040 (quasi zero), mentre l'ibrido resta a 0.480 – la differenza più importante misurata in tutto l'esperimento, e la ragione pratica per cui BM25 non è opzionale su corpus di queste dimensioni.
3. **La Faithfulness beneficia chiaramente del retrieval ibrido:** margini fra +0.094 e +0.197 in tutte le configurazioni, il risultato più stabile di tutto l'esperimento.

4. **La robustezza alle parafrasi non premia sempre l'ibrido:** solo a $N=1\,000$ l'ibrido è più stabile del base; a $N=5\,000-25\,000$ è vero il contrario. Aggiungere BM25 aiuta la precisione ma non garantisce automaticamente più stabilità a piccole variazioni della query.

I limiti principali riguardano il gold standard (definito manualmente e limitato a 5 query), il costo computazionale su CPU (necessario in assenza di GPU, e rilevante: l'intero esperimento su 5 configurazioni richiede diverse ore di generazione), e la faithfulness proxy che è una stima lessicale e non valuta la correttezza logica delle risposte.